Recombination in a Self-Modifying Network Search: Building Blocks, and the Weight Inheritance That Crossover Needs

Vasili Gavrilov*

2026

Abstract

Ordinarily a neural network must be handed its shape before it can learn: how many layers, how wide, which activation. The system studied here is handed only the training data and a target error, and finds the rest itself by evolution: its depth and width, whether each layer is dense or a weight-sharing convolutional one, and its own learning rate, momentum, and activation. Onto that mutation-driven search we add *crossover*, recombination of two parents, following the intuition that mixing building blocks should reach a good architecture faster than mutating one lineage. The paper follows that intuition to a resolved conclusion, in three experiments. First, on a problem with genuine separable building blocks, concatenated deceptive trap functions, our one-point crossover beats mutation decisively, its advantage *growing* with the number of blocks, the textbook fingerprint of recombination. Second, on the neural architecture search, crossover is on-par with mutation, no better; we diagnose why, with measurements, and the reason is structural. The architecture does have a heritable fitness, how fast it trains to the target, but on this task that trainability is a single entangled property, fungible in capacity and co-adapted with the hyper-parameters, with no independent sub-parts, so there are no building blocks for recombination to combine even though there is a trait to inherit. Third, we supply the missing ingredient, *weight inheritance*: a candidate is a modular set of sub-networks that carry their trained weights across generations, so a solved part is a self-contained transferable block. Now crossover wins again, and by a wide margin: on a task of several hard components it reaches the solution in about twelve generations regardless of component count, while mutation is several times slower and often fails outright. The building-block intuition holds for neural architecture search exactly when the genome carries the solution, not only its shape.

1 Introduction

Neural architecture search (NAS) automates a choice a human normally makes: the shape of the network. Its methods differ mainly in the search strategy: reinforcement learning [8], gradient-based relaxation [10], Bayesian optimization, random search, and evolutionary search [12]. Ours is evolutionary: the topology is evolved while each candidate's weights are trained by back-propagation [9]. Ordinarily the architecture and training knobs are arguments the user supplies, **predict(topology, activation, ...)**; the system here is handed only the data and a target error, **self_modifying_predict(data, error)**, and discovers the rest. The engine grew out of the author's 1997 seminar thesis, which derived the general multi-layer delta rule the training still uses [2].

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including rule engines and expert systems. Reference implementation: `https://github.com/Anode1/SMBPANN`.

A recurring result in this field is that *random search is hard to beat* [7, 11]. This paper asks whether *recombination* beats mutation at architecture search, and reaches a two-sided answer that resolves cleanly: crossover needs building blocks that carry a solution, standard NAS does not provide them, and adding weight inheritance does. The self-adaptive machinery and the evolution-strategy view we lean on are standard in the optimization literature the author has worked in [3].

2 Method

2.1 The self-modifying search

A network is a feed-forward stack of dense and convolution-like layers. For a dense layer ℓ the forward pass is $a^{(\ell)} = g(W^{(\ell)}a^{(\ell-1)} + b^{(\ell)})$, with g a per-network hidden activation (logistic sigmoid, tanh, or ReLU) chosen by the genome; the output layer is a dense sigmoid. A convolutional layer applies F shared kernels of width K across the previous layer, so its width is derived, $d_\ell = F(d_{\ell-1} - K + 1)$: the local, weight-sharing structure LeCun introduced for images [6], here a move the search can take. Weights are trained by back-propagation with momentum (the multi-layer delta rule of the 1997 thesis [2]), initialized uniformly in $[-2.4/f_i, 2.4/f_i]$. All arithmetic is 32-bit float; all randomness flows through a seeded xorshift generator.

A *genome* carries the topology (per hidden layer: dense or convolutional, and its free parameters) and its own training hyper-parameters (learning rate, momentum, activation), plus a self-adaptive mutation rate. One *mutation* makes a single small change: perturb a layer, flip a layer dense↔conv, or add or remove a layer. Reproduction is self-adaptive in the sense of evolution strategies [3]. The search keeps a population, retains the best few (elitism), and refills by reproduction, against a matched random-search control.

2.2 The crossover operator

To the mutation-only search we add optional *crossover* (`evolve -X`): a child is built from two elite parents by a one-point splice of their hidden-layer lists, a prefix of one and a suffix of the other, with a blend of their hyper-parameters, then mutated as usual. With `-X 0` the operator is inert and the run is byte-identical to the mutation-only search.

3 Experiments

3.1 The operator works: a building-block validation

We first check that the operator does what crossover should, on a problem built to have separable building blocks: concatenated deceptive trap functions, a genome of M blocks of K bits, where a block scores K only if all its bits are one, else $K - 1$ minus its number of ones, a deceptive gradient luring a bit-flipping search toward all-zero. The global optimum (all ones) is reachable only by assembling correctly-solved blocks. We race the same GA (tournament selection, elitism) with one-point crossover against mutation-only, over 200 seeds, scanning the block count (`make bbtest`).

Table 1 is decisive: at sixteen blocks mutation reaches the optimum on 28 of 200 seeds, crossover on 199. The operator and methodology are sound; when separable building blocks exist, crossover finds and combines them, and its advantage scales with the amount of structure.

3.2 On the architecture search, crossover does not help

We then run the same on/off comparison on the architecture search: for each seed, a fresh synthetic classification task ($d = 8$ inputs, a random-projection sine label), the self-modifying search

Table 1: Crossover versus mutation on concatenated deceptive traps (K = 4-bit blocks, population 1000, tournament 3, 200 seeds). As the number of building blocks grows, crossover's advantage grows: mutation's cost explodes and it begins to fail while crossover keeps solving.

blocks M (genome)	mutation-only	crossover + mutation	speedup
4 (16 bits)	9.2 gens, 200/200	5.1 gens, 200/200	1.8×
8 (32 bits)	70.8 gens, 200/200	14.2 gens, 200/200	5×
12 (48 bits)	257 gens, 185/200	22.6 gens, 200/200	11×, more reliable
16 (64 bits)	363 gens, **28/200**	42.3 gens, **199/200**	mutation fails

run twice from the identical initial population, once mutation-only and once with crossover, each until its best validation error reaches a target. Over the paired runs, crossover and mutation are indistinguishable: the same fraction reached the target, the same median generations, an even head-to-head. Adding recombination did nothing, and not because the operator is weak (Table 1).

3.3 Why: the genome carries architecture, not the solution

Two measurements explain it. First, architecture alone does not solve the task: trained with default hyper-parameters, every topology, dense or convolutional, scored 0.25–0.59 test-MSE, at or worse than chance. The *hyper-parameters* do the work, brittly: holding a small $8, 4, 1$ topology and varying only the learning rate, momentum, and activation swings the error from 0.09 to 0.65; with a good hyper-parameter set fixed, varying the topology is narrow and non-monotone ($8, 4, 1 \rightarrow 0.09$ while $8, 8, 1 \rightarrow 0.37$). The two dimensions are entangled and the landscape rugged, and crossover of a topology block breaks the co-adaptation that made it good. Second, a parallel-branch probe (`make modnas`, gradient-checked) shows that even a purpose-built modular architecture does not decompose, because *neural capacity is fungible*: one sufficiently-wide branch does what several specialised ones would, so there are no independent modules to recombine.

It is worth being precise about what the genome does carry, because it is not nothing. The architecture has a real, heritable fitness, namely how well it trains: how fast it reaches a target error, or the error it reaches in a fixed budget. That trainability is exactly what selection scores and what crossover could in principle recombine. The difference from the trap is not that the architecture genome is empty, it is that its one heritable trait does not *decompose*. In the trap the bits *were* the solution and split into independent blocks, so crossover combined solved blocks. Here the trainable-ness of a network is a single entangled property, fungible in capacity and co-adapted with the hyper-parameters, with no independent sub-parts each carrying their own share of it, so there are no building blocks to combine even though there is a trait to inherit. Weight inheritance changes this by giving the genome a *second* thing to carry, the trained solution itself, which in a modular network does split into transferable parts.

3.4 Weight inheritance restores the building blocks

That diagnosis names its own fix: let the genome carry the learned state, not only the shape. We test it in a modular setting (`make modevo`, back-propagation gradient-checked). A candidate is G independent sub-networks, one per output component; each component is a hard, high-frequency sine of its own input group, so training a sub-network from a random start succeeds only sometimes, a lottery. A candidate *carries its sub-networks' trained weights across generations* and fine-tunes them further, so a solved component is a self-contained, transferable block. Crossover recombines whole sub-networks (with their weights) from two parents; mutation must re-roll the initialization lottery for a stuck component within one lineage. We count generations to solve

Table 2: Weight-inheriting crossover versus mutation on the modular task (population 30, fine-tune 300 steps/generation, 60 search seeds, cap 120 generations). Crossover solves every task in about twelve generations whatever the component count; mutation is several times slower and, as components accumulate, frequently fails to solve at all within the cap.

components G	mutation (re-init)	crossover (inherit)
2	60/60, 31.0 gens	60/60, **11.4** gens
3	**20/60**, 47.9 gens	60/60, **12.1** gens
4	59/60, 84.6 gens	60/60, **12.6** gens
5	59/60, 39.4 gens	60/60, **12.5** gens

all components (mean validation MSE < 0.08), over 60 search seeds, scanning the number of components at fixed per-component difficulty.

Table 2 is the resolution. With weights inherited, crossover wins again and by a wide margin: it solves 60/60 tasks in roughly twelve generations independent of the number of components, while mutation is 2.7 to 6.7× slower when it succeeds and unreliable when components accumulate (at $G = 3$ it solves only a third). The same recombination that did nothing to an architecture-only genome (Section 3.2) is decisive once the genome carries the solution, exactly as it was on the trap bits. This is the author's intuition made concrete: a useful combination that evolution has found is inherited and keeps evolving, rather than being retrained from zero, which is both what gives recombination a solution to transfer and what makes the search converge faster.

3.5 Weight inheritance in the full engine, and its budget dependence

Table 2 is a controlled modular probe. We also wired weight inheritance through the full self-modifying engine (`evolve -w`): a checkpoint of each candidate's trained weights crosses the worker boundary as a plain-text file, and a child warm-starts from its parent's checkpoint over the layers a mutation left unchanged, while the elite resume their own training each generation. With inheritance off the run is byte-identical to before; the matched random control, which has no lineage to inherit from, is identical in both arms. Because warm-starting saves training a from-scratch search discards, the gain should be largest when the per-candidate epoch budget is small, and fade as candidates converge on their own. We sweep it (`scripts/inherit_sweep.sh`): a paired A/B over 300 fresh-task seeds per cell, the search run once from scratch and once with inheritance at a fixed budget.

Table 3 shows the predicted gradient cleanly: inheritance lowers the search's final error at equal compute across the board, but the effect falls from about a fifth of the error at a 20-epoch budget to a few percent at 160, where each candidate largely trains to convergence on its own and there is little discarded training to reclaim. More generations help as well, since an elite lineage accumulates training over more rounds. The engine result and the modular probe agree: carrying the learned state, not only the architecture, is what lets an evolutionary search stop paying to relearn what it already found.

3.6 Where architecture is decisive

The whole account rests on architecture being fungible on our tasks; it is worth showing the opposite regime exists, where architecture is decisive and a search would have real structure to find. We built a translation-invariant image task (`make conv2d`): a small diagonal motif, "\" or "/", placed at a random position in a noisy 8×8 image, the label being its orientation. A single 2D convolution that detects the motif wherever it sits (gradient-checked, 0/40 finite-difference

Table 3: Weight inheritance in the engine: mean final validation error, from scratch versus inherited, over 300 paired seeds per cell ($d = 6$, pop 10, ≤ 3 layers of width ≤ 12). Inheritance wins significantly in every cell (exact sign test $p < 10^{-3}$), but the relative error reduction shrinks monotonically as the per-candidate epoch budget grows, from 21% at 20 epochs to 6% at 160: warm-starting helps most when training is scarce.

epochs	generations	from scratch	inherit	inherit wins (/300)
20	8	0.0730	0.0609	242
20	20	0.0660	0.0518	251
40	8	0.0646	0.0569	218
40	20	0.0576	0.0474	239
80	8	0.0603	0.0539	207
80	20	0.0527	0.0464	218
160	8	0.0568	0.0534	176
160	20	0.0499	0.0454	199

Table 4: An image task where architecture is decisive: test error rate on the translation-invariant motif task, a 2D convolution (6 filters) versus a dense network (24 hidden), mean of 8 seeds. With limited data the convolution generalizes and the dense network is near chance; only with an order of magnitude more data does the dense network catch up.

training images	2D convolution	dense network
2000	**0.001**	0.388
6000	**0.000**	0.192
20000	0.000	0.002

mismatches) generalizes; a dense network of similar size, which must relearn the motif at every position, does not, until it is given far more data.

Table 4 shows the contrast: at 2000 images the convolution is at 0.1% error and the dense network near chance (39%). Here architecture is not fungible, the right structure carries the solution and the wrong one cannot reach it with limited data. This is the regime in which architecture search, and recombination of architectural building blocks, would pay, and it is the natural next home for the methods this paper studied.

4 Discussion

The three experiments compose into one statement. Crossover's building-block advantage is real and provable (Table 1), but it applies only when the genome carries a heritable trait that *decomposes* into independent parts. Standard neural architecture search does have a heritable trait, an architecture's trainability, which selection scores as time-to-target or limited-budget error; but on a fungible-capacity, hyper-parameter-entangled landscape that trait is a single entangled quantity with no independent sub-parts, so recombination has no blocks to combine, which is a structural reason random search is a strong NAS baseline and the field [9] routinely drops crossover. Weight inheritance gives the genome a second, decomposable thing to carry, the trained solution, and in a modular network the advantage returns in full (Table 2). The lesson is not that crossover is weak or strong in the abstract, but that it is exactly as useful as the genome is a carrier of separable building blocks.

Limitations. Scale is small and tasks synthetic; Table 2 is a controlled probe with a fixed per-component task and a fixed module count, and the engine sweep (Table 3) is a 6-input

problem with shallow networks. Mean squared error stands in for classification error. None of this is hidden.

5 Conclusion and future work

Handed only the problem and a target error, the search discovers architecture, layer type, and training hyper-parameters. Recombination does not speed the architecture search while the genome carries only shape, and we now understand exactly why and exactly when it does: crossover needs building blocks that carry a solution, and weight inheritance supplies them (Table 2). That inheritance is now wired through the engine as well (`evolve -w`): children warm-start from their parents' checkpointed weights, and the search reaches a lower error at equal compute. We also showed the regime these methods are built for exists: on a translation-invariant image task a 2D convolution is decisive where a dense network fails (Table 4), the setting in which architecture search and its recombination of building blocks would pay. The clear remaining step is to run that search there, and to bridge to a standard tabular NAS benchmark.

Reproducibility

Every result is regenerated from [13] with a single command. The engine builds with `make` (C99, no dependencies) into `smbpann` (worker), `evolve` (search and control, with `-X` for crossover), and `gentask`; `make ut` runs the unit and gradient-check suite under AddressSanitizer and UBSan. The three validation probes are standalone: `make bbtest` (Table 1, building-block traps), `make modnas` (the parallel-branch gradient check), and `make modevo` (Table 2, weight-inheriting crossover; run `./modevo G freq`); and `make conv2d` (Table 4, the image task where architecture is decisive). The neural crossover A/B is `scripts/xover_ab.sh`; weight inheritance in the engine is `evolve -w 1` (the worker's `-W`/`-w` save and warm-start checkpoints), its A/B is `scripts/inherit_ab.sh` and the budget sweep of Table 3 is `scripts/inherit_sweep.sh`. Each run uses a fixed 32-bit xorshift generator, so the numbers are identical on re-running.

References

[1] D. E. Rumelhart, G. E. Hinton, R. J. Williams. Learning representations by back-propagating errors. *Nature* 323, 1986.

[2] V. Gavrilov. *Backpropagation Feed-Forward Neural Networks.* Seminar in Artificial Intelligence, Open University of Israel, 1997 (revised digital edition, Zenodo 2026). `https://doi.org/10.5281/zenodo.20450526`

[3] I. Rechenberg. *Evolutionsstrategie.* 1973; H.-P. Schwefel, *Numerical Optimization of Computer Models*, 1981.

[4] J. H. Holland. *Adaptation in Natural and Artificial Systems.* University of Michigan Press, 1975 (schemata and building blocks).

[5] K. Deb, D. E. Goldberg. Analyzing Deception in Trap Functions. *Foundations of Genetic Algorithms* 2, 1993.

[6] Y. LeCun, B. Boser, J. S. Denker, et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.

[7] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.

[8] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.

[9] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.

[10] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.

[11] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019.

[12] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.

[13] V. Gavrilov. SMBPANN: a self-modifying backpropagation feed-forward neural network. Source code, `https://github.com/Anode1/SMBPANN`.